OmniSearch

RL Training Experiments (HAPPO)

A record of the reinforcement-learning training runs for the wildfire search-and-rescue scenario: what was tried, episode budgets, training hours, results, and comparison against the non-learning baselines.

Author: Oleksii Lavrenin (seniorQAautomationEngineer). A formatted PDF of this report lives at RL_TRAINING_EXPERIMENTS.pdf.

Goal & question

Can a trained MARL policy (HAPPO) match or beat hand-coded coordination heuristics on the mission metric **survivor recall** (fraction of 5 survivors confirmed by a ground robot)? Drones scout (broad downward camera); ground robots confirm (precise, slow). Three hand-coded baselines provide the bar.

Evaluation conditions (all numbers below)

- **Terrain:** real cached terrain, ~1 km bbox (`data/terrain_cache/malibu_creek_1km_128.npz`), 128×128 grid.
- **Episode:** 1000 steps; `recall = found / 5` at episode end; mean over 5–10 seeds.
- **Reward-signal floors (opt-in, default 0.0):** `drone_min_footprint = 0.15`, `ground_confirm_min = 0.20`. On a real-km terrain the *physical* drone footprint (~0.033) and confirm range (~0.017) shrink so far that neither RL nor the baselines get a usable scout/confirm reward; the floors restore it. **Both HAPPO and the baselines are evaluated at the same floors** — comparisons are apples-to-apples. Repo defaults remain 0.0 (physical). *Results are only comparable across the team at the same floor settings.*

Non-learning baselines (the bar to beat)

Fixed hand-coded heuristics — no training. (Measured at floors 0.15 / 0.20, 1 km, 1000 steps.)

Baseline	How it works	scouted	recall
random	everyone moves randomly (control)	0.62	0.06
nearest_candidate	random drones; UGVs → nearest scouted survivor	0.60	0.62
lawnmower	serpentine drone sweep; UGVs A* to nearest scouted	0.82	0.92

HAPPO training experiments (chronological)

Each run diagnosed and fixed a distinct bottleneck. Episodes × 500 steps = env steps. * = short diagnostic run.

#	Episodes	hrs	Key configuration	scouted	recall	Finding / bottleneck
1	100	0.1	default 22 km terrain, no floors	0.2	0.07	Wrong (huge) terrain + no reward signal
2	1000	0.6	1 km, footprint floor 0.15	0.33	0.00	Cost-dominated reward → do-nothing optimum
3	2000	1.1	+entropy 0.08, search-reward	0.16	0.00	Reward-hacks dense shaping; action saturation
4	300*	0.2	+std_y 1.0, +entropy 0.25	0.47	0.00	Action saturation **broken**; scouting recovers
5	3000	2.0	std1.0 + ent0.25 + search	0.56	0.00	Scouting up; ground robots never confirm
6	4000	2.6	+ground_confirm floor 0.12	0.36	0.16	Recall lifts off 0 for the first time
7	10000	8.6	same (non-recurrent)	0.33	0.10	**Plateau** — more budget does NOT help
8	6000	5.2	+coverage reward +recurrent (GRU)	0.62	0.12	Coverage breakthrough; scouting now competitive
9	300*	0.2	+ground approach reward, confirm 0.20	0.56	0.20	Ground-confirm leg lifts recall to 0.20
10	6000	5.9	all fixes (coverage+recurrent+approach)	0.56	0.20	Plateaus at 0.20

Total HAPPO training ≈ 31 hours across all runs (above, plus earlier 80k/400k experiments), plus behaviour-cloning / DAgger / evaluation. On CPU (Apple M-series): ~150–280 env-steps/sec; recurrent (GRU) runs ~30 % slower.

Learning from demonstration (cloning the lawnmower expert)

Pure RL plateaus at recall ~0.20. To approach the heuristic, clone its behaviour, then RL-fine-tune.

Approach	scouted	recall	Note
Behaviour cloning (feedforward), no RL	0.74	0.30	Clones the sweep well; ground-confirm precision doesn't transfer
BC + RL fine-tune	0.46	0.36	RL erodes the cloned sweep
Recurrent BC (GRU, BPTT)	**0.74**	**0.38**	**Best learned** — memory captures multi-step navigation
DAgger (clone precision, 8 iters)	0.56	0.34	Improves over feedforward BC (0.30); below recurrent BC

Comparison: learned vs. non-learning

Policy	Trained?	recall
HAPPO — pure RL (best, all fixes)	yes	0.20
HAPPO — BC + RL fine-tune	yes	0.36
HAPPO — recurrent BC (best learned)	yes	**0.38**
random	no	0.06
nearest_candidate	no	0.62
lawnmower	no	0.92

Conclusion (honest)

- **HAPPO improved from a do-nothing collapse (0.00) to a functioning searcher** — best learned recall **0.38** (recurrent BC). Drone scouting became competitive with the baselines (0.56–0.74 vs lawnmower 0.82).
- **It does NOT beat the hand-coded baselines** (nearest 0.62, lawnmower 0.92). Pure RL plateaus at ~0.20 — more episodes do not help (1k and 10k both ~0.10–0.20). Behaviour cloning lifts a learned policy to 0.30–0.38.
- **Why:** six bottlenecks were diagnosed and fixed (footprint floor, action-saturation, coverage reward, recurrent memory, ground-confirm floor, ground-approach reward). The residual gap is the ground robots' A* approach-to-survivor **precision**, which neither RL nor feedforward BC reproduces. Recurrent BC and DAgger target exactly this.

- **Scientific takeaway:** hand-coded coordination heuristics remain superior on mission recall; learned MARL matches the individual *scouting* sub-behaviour but not the full *scout* → *confirm* coordination at this budget.

Addendum — Floor-0 investigation (no detection floor, physical sensors only)

The runs above use opt-in detection **floors** (`drone_min_footprint = 0.15`, `ground_confirm_min = 0.20`) to make survivors detectable. A follow-up study imposed a hard constraint: **the floor must stay 0** — only *physical* variables (terrain size, camera FOV, flight altitude, episode length, sensor range in meters) may change. Everything below is at **floor 0** and is therefore **not directly comparable** to the floored numbers above.

Root cause of 0 recall at floor 0 (geometry, not the algorithm)

Detection geometry is physical: the drone scout footprint is `flight_altitude · tan(FOV/2)` and the ground confirm radius is `ground_confirmation_range_m · sim_units_per_meter`, all in simulation units on a fixed `[-1, 1]²` map. `sim_units_per_meter` is set by the terrain's real-world extent:

Terrain	extent	`sim_units_per_meter`	confirm radius @10 m	footprint @50 m
Malibu Creek **State Park** (default)	~22 km	9.1e-05	0.0009 (~0.04 % of map)	0.0045
Malibu Creek **small**	~3 km	6.6e-04	0.0066	0.033
Malibu Creek **1 km**	~1.2 km	1.7e-03	0.017	0.083 (~4 % of map)

On the default park terrain a survivor is a ~0.04 %-of-map pinpoint at floor 0 — **even the hand-coded experts score 0.00** there. It was never an algorithm problem; it was the map scale. Diagnostics: `scripts/diag_terrain_floor0.py`, `scripts/diag_floor0_ceiling.py`.

The fix that keeps floor 0: smaller terrain + stronger physical sensors

On the **1 km** terrain with wider FOV (140°), higher flight (50/80/100 m) and longer episodes (1000 steps), expert recall recovers to **0.47–0.60** at floor 0 — real, learnable signal. Packaged as `scripts/train_happo_smoke.py --preset floor0-1km` (sets terrain, sensors, recurrent policy, confirmation-dominant reward, coverage observation; floor stays 0). Eval: `scripts/eval_floor0_1km.py`.

Results at floor 0 (1 km, 1000 steps, 3 seeds)

Approach	recall @0.0	recall @0.3 dropout	UGV travel
HAPPO from scratch (240k)	0.07	0.07	~2.9
HAPPO more compute (3 M)	0.07	0.00	~2.9
BC clone alone	0.07	—	—

Approach	recall @0.0	recall @0.3 dropout	UGV travel
BC → RL fine-tune (1.2 M)	0.00	0.07	~2.9
+ team-coverage observation (obs 54→91)	0.00	0.00	2.86
+ confirmation-dominant reward	**0.10**	0.00	2.48
+ ground exploration reward	0.07	0.00	2.30
lawnmower expert	**0.60**	0.47	4.78
nearest_candidate expert	**0.47**	0.13	4.81

Conclusion (floor 0)

- The **environment bottleneck is solved**: at floor 0 the experts hit 0.47–0.60 once the terrain scale and physical sensors are right (previously *everything*, experts included, was 0.00).
- **RL plateaus at ≤ 0.10.** Across seven configurations — more compute, behaviour cloning, BC+RL, a coverage observation, a confirmation-dominant reward, and an explicit ground-exploration reward — the trained **ground robots refuse to sweep**: UGV travel stays ~2.3–2.9 vs the experts' ~4.8, even after movement costs were cut to 0 and movement was directly rewarded. This is an **optimization/coordination pathology** (low-velocity action collapse), not a reward-design gap.
- **Recommendation:** at floor 0, deploy the hand-coded experts (0.60 recall) and treat learned MARL coordination as open research. The contribution here is the diagnosis + the reproducible floor-0 harness, not a learned policy that beats the heuristics.

Sensor-sensitivity sweep — what 90% recall actually costs (floor 0)

Recall at floor 0 is dominated by *physical sensor generosity*, not by the algorithm. Holding the floor at 0 and pushing the physical sensors far beyond realistic values makes the search nearly trivial — and HAPPO then reaches ~0.90. This is reported as an **upper bound / sensitivity result**, not as the headline, because the sensors are not physically plausible for the platform.

Sensor config (floor 0, 1 km, 1000 steps)	FOV	flight	confirm range	HAPPO recall	realism
Realistic (headline)	90°	50/80/100 m	60 m	learned ≤ 0.10; experts 0.47–0.60	plausible camera + UGV
Wide-sensor	140°	50/80/100 m	30 m	~0.07–0.10	borderline
Generous	170°	120/180/200 m	300 m	**0.47**	implausible

Sensor config (floor 0, 1 km, 1000 steps)	FOV	flight	confirm range	HAPPO recall	realism
Very generous (upper bound)	170°	120/180/200 m	**600 m**	**0.90** (0.95 @0.3 dropout)	confirm range ≈ ½ the map; trivializes search

At 600 m confirm range on a ~1.2 km map, a single UGV "confirms" survivors across roughly half the map without traveling — the coordination/search problem the project is about has effectively been removed. The 0.90 number is therefore a measure of *how easy generous sensors make the task*, not of learned search competence. Checkpoint: `happo_c600_floor0_1km`. Diagnostic that finds the generous configs at which experts hit ~0.90: `scripts/diag_floor0_generous.py` (fixed at `n_ground = 2`).

The exported viewer run (`web/trajectories/happo_trained.json`, seed 0) makes this concrete: the 600 m confirm range is **0.99 in sim units on the `[-1, 1]²` map** (radius ≈ 1), i.e. one UGV's confirm disc nearly spans the whole map. That run hits `survivor_recall = 1.0` by **step 36** with **UGV travel ≈ 0.38** (vs experts' ~4.8) — the robots barely move and still find everyone. That is the definition of a trivialized search, and exactly why 90% is reported as an upper bound, not a result.

Confirmation realism — line-of-sight collapses the 90% (no retraining)

The 0.90 is also propped up by a *second* unrealistic assumption: confirmation was a pure **proximity** check (`dists < confirm_range`), so a ground robot "confirms" a survivor even through a mountain. Adding `confirm_requires_los=True` keeps the same 600 m range but additionally requires an **unobstructed terrain sight line** (eye→target ray not blocked by intervening elevation). Evaluating the *same* `happo_c600_floor0_1km` checkpoint, 3 seeds × 1000 steps, floor 0:

confirmation rule	HAPPO recall	lawnmower	nearest	HAPPO UGV travel	HAPPO TTV
proximity only (600 m)	**0.93**	1.00	1.00	2.45	62.8
proximity + **line-of-sight** (600 m)	**0.40**	0.73	0.73	4.04	248.8

Just requiring the robot to actually *see* the survivor drops learned recall **0.93 → 0.40** and forces the UGVs to move (travel 2.45 → 4.04, TTV 4× longer) — the "confirm while standing still" behaviour disappears. (HAPPO falls further than the experts because it was trained without the LOS constraint and never learned to reposition for a clear view; the experts adapt.) Combined with a *realistic* range, recall would be lower still. Flag: `--confirm-requires-los` on `scripts/eval_floor0_1km.py`; env kwarg `confirm_requires_los` (+ `confirm_observer_height_m`, `confirm_target_height_m`, `confirm_los_samples`).

The realistic path to ≥0.9 — drone-assisted confirmation (not sensor cheating)

Ground-only confirmation cannot reach 0.9 under realistic line-of-sight, even with a long ground range and unlimited time (the 2 UGVs physically can't reach + see every survivor through terrain). Measured ceilings (LOS on, FOV 90°, n_ground=2, 6 envs):

confirmation model	sensors	best expert recall
ground-only	60 m range	~0.30 (flat vs episode length)
ground-only	200 m	0.47–0.57
ground-only	400 m	0.53–0.63
ground-only	600 m	0.57–0.73
+ drone EO/IR confirmation (FOV 90°, 50–100 m)	60 m ground	**0.83**
+ drone EO/IR confirmation (FOV 120°, 80–140 m)	60 m ground	**1.00**

The honest way to ≥0.9 is to make the *model* more realistic, not the sensors more generous: real wildfire-SAR drones carry EO/IR (thermal) cameras and detect/confirm people **from altitude**. Enabling `drone_can_confirm` lets a drone confirm a survivor inside its camera footprint with a clear **top-down** sight line (terrain rarely occludes a steep downward view). This keeps every realism constraint — floor 0, realistic FOV/altitude, line-of-sight for both air and ground, `n_ground=2` — and the hand-coded expert reaches **0.83 (FOV 90°) to 1.00 (FOV 120°)**. This reframes the mission as it actually works: **drones detect & confirm from the air; the 2 UGVs reach and verify a subset on the ground.** Env kwargs: `drone_can_confirm`, `r_drone_confirm`; diagnostic: `scripts/diag_realistic_ceiling.py --drone-confirm`.

A learned HAPPO policy reaches ≥0.9 (BC warm-start + RL fine-tune)

The drone-confirm result above is honest, but initially it was only the *hand-coded expert* that reached ≥0.9 — a from-scratch HAPPO run under the same realistic config plateaued at **0.50–0.70** (1.2M steps, `happo_realistic_droneconfirm`). Pure RL learns the sub-behaviours but not the near-optimal sweep-and-confirm coordination.

Closing the gap took two steps:

1. **Behaviour-clone the lawnmower expert** (recall 1.0 under this config) into all 5 HAPPO actors — `scripts/train_bc_happo.py` with the realistic flags (`--coverage-obs-grid 6 --confirm-requires-los --drone-can-confirm`, FOV 110°, floor 0). Recurrent BPTT clone, NLL 0.08 → −1.3.
2. **RL fine-tune from that warm-start** — `train_happo_smoke.py --model-dir results/bc_happo_realistic` (1M steps). The fine-tune *starts* at ~79 episode reward vs ~42 from scratch.

Measured recall of the **learned** policy (floor 0, LOS on, drone-confirm, FOV 110°, `n_ground=2`, 1000-step episodes, best checkpoint `results/happo_realistic_best`):

policy	comms dropout	recall	hazard	UGV travel
HAPPO from scratch (1.2M)	0.0	0.60	—	—
HAPPO BC + RL fine-tune	0.0	**0.97**	22	1.38

policy	comms dropout	recall	hazard	UGV travel
HAPPO BC + RL fine-tune	0.3	**0.97**	11	0.95

This is a **learned MARL policy at 0.97 recall**, robust to 30 % comms dropout — not a scripted heuristic. Note the late-training regression seen before (final checkpoint fell to 0.80); periodic checkpoint snapshotting + recall-based selection keeps the best (~0.93–0.97 mid-run) policy.

Honest headline

- **Realistic sensors, floor 0, ground-only confirmation:** experts 0.47–0.60, learned MARL ≤ 0.10.
- **Realistic sensors, floor 0, with drone EO/IR confirmation (the realistic SAR model):** experts 0.83–1.00, and a **learned HAPPO policy (BC warm-start + RL fine-tune) reaches 0.97** (robust under 0.3 comms dropout) — ≥0.9 is achievable *honestly* by both the heuristic and a trained policy, by letting drones confirm from altitude rather than inflating sensors. This is the credible high-recall result.
- **90% is achievable only by making the sensors unrealistically powerful** (confirm range ≈ half the map) **and** by letting confirmation pass through terrain. Requiring line-of-sight alone drops it to 0.40. Report 90% explicitly as a sensor-sensitivity upper bound, never as the operating point.
- Map size is a difficulty knob in the *opposite* direction: a larger map shrinks `sim_units_per_meter` and pushes recall back toward 0 (the original park-terrain failure). Keep the 1 km map.

Viewing a true (realistic-sensor) search

Export baselines under realistic sensors so the viewer shows a genuine partial-view sweep (footprint ≈ 16 % of map radius), not an instant solve:

```
python scripts/export_trajectories.py --approach all --skip-happo-manifest \
  --terrain-cache-path data/terrain_cache/malibu_creek_1km_128.npz --grid-size 128 \
  --drone-camera-fov-deg 90 --drone-flight-levels-m 50 80 100 \
  --ground-confirmation-range-m 60 --steps 600
python -m http.server -d web   # open http://localhost:8000
```

`--skip-happo-manifest` prevents the generous-sensor checkpoint's config from overriding the realistic flags; `--ground-confirmation-range-m` sets the physical confirm range (not a floor).

New tooling added for this study: `scripts/diag_terrain_floor0.py`, `scripts/diag_floor0_ceiling.py`, `scripts/eval_floor0_1km.py`, `train_happo_smoke.py --preset floor0-1km` (+ flags `--coverage-obs-grid`, `--reward-confirm`, `--n-rollout-threads`, sensor flags); env additions `coverage_obs_grid`, `r_pending_penalty`, `r_ground_coverage`/`ground_coverage_radius`; `scripts/diag_floor0_generous.py`; export flags `--skip-happo-manifest`, `--ground-confirmation-range-m`.

Critical changes that reach ≥0.9 detection

The full investigation above traces how survivor **detection (recall)** went from a do-nothing 0.0 to a credible ≥0.9. Distilled to the changes that actually matter:

Rank	Change	Commit	Why it's critical
1 (decisive)	`drone_can_confirm` — **aerial EO/IR confirmation**	`026d5c3`	The single change that delivers ≥0.9 honestly. A drone confirms a survivor inside its camera footprint with a clear top-down sight line. Expert recall jumps to **0.83 (FOV 90°) → 1.00 (FOV 120°)**; realistic ground-only confirmation tops out at ~0.3–0.7.
2 (precondition)	**1 km terrain scale** (floor-0 harness)	`f20d9f2`	Necessary enabler. On the default ~22 km map a survivor is a ~0.04 %-of-map pinpoint and **everyone scores 0.0** — the geometry, not the policy, was the blocker. The 1 km map (`sim_units_per_meter` ~20× larger) makes confirmation physically possible.
honesty gate	`confirm_requires_los` **— line-of-sight**	`cf67347`	Does **not** raise recall; it *lowers* the cheat. Exposes that the generous-sensor 0.9 was "confirm through a mountain" (0.93 → 0.40). Critical for proving the 0.9 you keep is real.
learned-policy lever	**BC warm-start + RL fine-tune**	BC (`train_bc_happo.py`) → `train_happo_smoke.py --model-dir`	Gets a *learned* HAPPO policy to ≥0.9, not just the expert. Cloning the 1.0-recall lawnmower then RL-fine-tuning yields **0.97 recall** (robust to 0.3 comms dropout); from-scratch RL under the same realistic config plateaus at 0.50–0.70.
supporting	coverage observation, confirmation-dominant reward, idle penalty, ground-exploration reward, GRU policy	`f20d9f2`	Training-side shaping. Necessary for the BC clone + fine-tune to match obs space and learn the sweep; on their own (from scratch) they reach only ~0.20–0.70.

One-line answer: the decisive change is `drone_can_confirm` (commit `026d5c3`) — letting drones confirm from altitude — made possible by the **1 km terrain** fix (`f20d9f2`) and validated as honest by the **line-of-sight gate** (`cf67347`). It is one extra OR-branch in the confirmation logic (`envs/wildfire_search.py`):

```
confirmed_by_ground = eligible_ground_confirmations.any(dim=1)

if self.drone_can_confirm:
    # Drone confirms a survivor in its camera footprint with a clear top-down LOS.
    drone_conf = drone_seen
    if self.confirm_requires_los:
        drone_conf = drone_conf & self._drone_confirm_los_mask(drone_pos, surv_pos)
    confirmed_by_drone = drone_conf.any(dim=1)

# A survivor counts as found if EITHER a ground robot OR a drone confirms it.
found = confirmed_by_ground | confirmed_by_drone
```

What was NOT critical: the generous 600 m ground confirm range gives 0.90–0.95, but that is a **sensor-cheat upper bound** (confirm range ≈ half the map) that collapses to 0.40 under line-of-sight. Report it only as a sensitivity result, never the operating point.

Code & commits

Training scripts:

- `scripts/train_happo_smoke.py` — HAPPO via HARL. Flags: `--terrain-cache-path`, `--drone-min-footprint`, `--ground-confirm-min`, `--entropy-coef`, `--reward-search`, `--recurrent`, `--model-dir`.
- `scripts/train_bc_happo.py` — behaviour cloning (feedforward + `--recurrent` BPTT) from the lawnmower.
- `scripts/train_dagger_happo.py` — DAgger (iteratively re-label the clone's own states with the expert).
- MAPPO/IPPO smoke scripts exist (BenchMARL) but their checkpoint→eval loaders are not wired, so they are not in the comparison.

Key commits:

- `34f27b6` — Add behaviour cloning from lawnmower + RL fine-tune for HAPPO
- `dcd92e1` — Add ground directed-approach reward for the scout→confirm leg
- `384fa70` — Add coverage reward + recurrent policy: HAPPO learns systematic search
- `1c62f87` — Make HAPPO trainable: opt-in floors, anti-saturation, search reward
- `122acda` — Floor drone scout footprint so survivors are detectable on real terrain

Note: all recall numbers are measured at the opt-in floors (0.15 / 0.20), not the physical defaults (0.0). Standardize the floor values across the team for comparable results.

TensorBoard (HAPPO)

HARL already writes TensorBoard events for HAPPO runs. OmniSearch now surfaces the log path in training output and in `train_happo()` return values (`tensorboard_log_dir`, `tensorboard_cmd`).

Single run

Run HAPPO training:

```
python scripts/train_happo_smoke.py --research --preset tuned
```

The script prints:

- TensorBoard log directory
- A ready-to-run command like:

```
tensorboard --logdir "results/harl_runs/.../logs" --port 6006
```

Open:

```
http://localhost:6006
```

All HAPPO runs

To browse all HARL/HAPPO experiments at once:

```
tensorboard --logdir "results/harl_runs" --port 6006
```

Tuning runs

`scripts/tune_happo.py` records each trial's training metadata in the results JSON (`results/happo_tuning_*.json`), including the checkpoint/manifest path under `train_result`. Use those paths to locate corresponding HARL run dirs and inspect them in TensorBoard.